

RAPPORT TECHNIQUE SIMULATEUR MARITIME

ROSILIA CABRELLE MODJO ABONA
ICAM SITE DE NANTES
35, avenue du Champ de Manœuvre 44470 CARQUEFOU



Table des matières

.....	0
INTRODUCTION.....	3
I. ARCHITECTURE GENERALE DU SIMULATEUR	4
1. Choix techniques et méthodologie	4
2. Architecture logicielle	4
II. DESCRIPTION DE L'INTERFACE UTILISATEUR	5
1. Bandeau supérieur commun aux deux onglets	5
2. Onglet « NRCS Simulation »	5
a. Section paramètres (à gauche).....	6
b. Section affichage et résultats (à droite)	6
c. Boutons.....	7
3. Onglet « Sea Surface Generation »	7
a) Paramètres	7
b) Affichage	8
c) Boutons.....	8
III. IMPLEMENTATION DES FONCTIONNALITES CLES.....	8
1. Calcul de la NRCS : Principe et Structure de calculerCallback.....	8
2. Superposition des courbes : superposerCallback	12
3. Export des données.....	13
4. Génération de surfaces de mer : generateSeaSurfaceCallback	13
IV. VALIDATIONS ET TESTS	14
1. Validation scientifique.....	14
2. Tests fonctionnels	15
V. AMELIORATIONS ET LIMITATIONS.....	15
1. Limitations actuelles	15
2. Perspectives à court et moyen terme	15
CONCLUSION	16
ANNEXES	17
Annexe A.....	17
Annexe B : Tableau récapitulatif des modèles implémentés.....	17
Annexe C : Glossaire.....	19
REFERENCES.....	21

Table des illustrations

Figure 1: Onglet NRCS	5
Figure 2: Onglet SeaSurface	7
Figure 3: Menu d'export- onglet NRCS	13
Tableau 1: Détails des modèles implémentés	17

INTRODUCTION

Ce rapport présente le développement complet d'un simulateur radar monostatique dédié à la surface de mer, réalisé en MATLAB. L'outil permet le calcul de la section efficace radar normalisée (NRCS) selon 22 modèles physiques, semi-empiriques et empiriques couvrant les bandes de fréquences UHF à W-band, ainsi que la génération de surfaces océaniques 2D basées sur le spectre d'Elfouhaily.

L'interface graphique, conçue avec la GUI Layout Toolbox à installer via le *Add-Ons manager* de MATLAB ou depuis [GUI Layout Toolbox - File Exchange - MATLAB Central](#) pour une modularité et une flexibilité maximale, intègre pour l'instant deux onglets fonctionnels, une gestion dynamique des paramètres, une superposition avancée de courbes avec suppression interactive, des exports multiples et une optimisation mémoire rigoureuse notamment lors de la génération de surfaces de mer.

I. ARCHITECTURE GENERALE DU SIMULATEUR

1. Choix techniques et méthodologie

Le simulateur a été entièrement développé en MATLAB pour profiter de ses capacités graphiques natives et de sa facilité de prototypage. Le choix de ne pas utiliser App Designer a été délibéré : il permet une compatibilité avec des versions MATLAB plus anciennes et une maîtrise totale de la mise en page grâce à la **GUI Layout Toolbox** (composants uix.VBox, uix.HBox, uix.Grid) disponible sur [GUI Layout Toolbox - File Exchange - MATLAB Central](#).

Le développement a suivi une approche itérative et incrémentale :

- **Phase initiale** : conception de l'interface de base et intégration de quelques modèles développés au sein du laboratoire (CMOD5, NRL, TSC)...
- **Phase d'enrichissement** : ajout progressif de modèles physiques avancés (TSM, SSA1, puis GO1sh et SPM1) et des paramètres associés (fetch, K_0/K_c)...
- **Phase de débogage graphique** : résolution longue et fastidieuse des problèmes d'affichages graphiques.
- **Phase d'optimisation** : implémentation d'une détection multi-plateforme de la RAM disponible, blocage de génération de la surface si dépassement de 10 %, ajout du label dynamique longueur d'onde, légende cliquable pour suppression de courbes...
- **Phase finale** : tests, documentation interne, amélioration de l'ergonomie (polices agrandies, tooltips)...

2. Architecture logicielle

Le simulateur est un script unique *SeaRadarSimulator.m*, exécutable directement. Il crée une *uifigure* contenant :

```
% Création de la figure principale
fig = uifigure('Name', 'Sea Surface Radar Simulator', 'Position', [100 0 1000
700]);
%fig.WindowState = 'maximized';

% Ajout du handle de la figure à app après sa création
app.handles.fig = fig;
app.handles.curves = struct('theta', {}, 'nrcs_db', {}, 'params', {}, 'color',
{}); %structure pour stocker les courbes

% Layout principal : Onglets pour NRCS et Surface de Mer
tabGroup = uitabgroup('Parent', fig, 'Position', [0 0 1000 700]);
tabNRCS = uitab('Parent', tabGroup, 'Title', 'NRCS Simulation'); % Premier
onglet
tabSeaSurface = uitab('Parent', tabGroup, 'Title', 'Sea Surface Generation'); %
Deuxième onglet
```

- Un *uitabgroup* avec deux onglets.
- Une structure globale *app.handles* stockant tous les composants UI et les données calculées (courbes NRCS, surface courante).
- Une série de fonctions locales (callbacks, utilitaires) pour la modularité.

Aucun fichier externe n'est requis hormis les fichiers logos et, pour le modèle SSA1, les fichiers de fonctions de corrélation précalculées.

NB : Bien que pas obligatoire pour le lancement initial de l'interface, les fonctions de calcul des modèles de NRCS sont nécessaires pour le bon fonctionnement de l'interface.

II. DESCRIPTION DE L'INTERFACE UTILISATEUR

1. Bandeau supérieur commun aux deux onglets

Le bandeau orange occupe les 50 premiers pixels en hauteur. Il contient :

- Un titre principal en police 22 pt : « Sea Surface Radar Simulator ».
- Les logos ICAM et IETR chargés dynamiquement via *imread*. La fonction *resizeLogos* (appelée à la création et via *SizeChangedFcn*) calcule l'échelle nécessaire pour que les logos occupent 75 % de la hauteur disponible tout en conservant leurs proportions. Un mécanisme de fallback affiche un texte « ICAM | IETR » si les images sont absentes.

2. Onglet « NRCS Simulation »

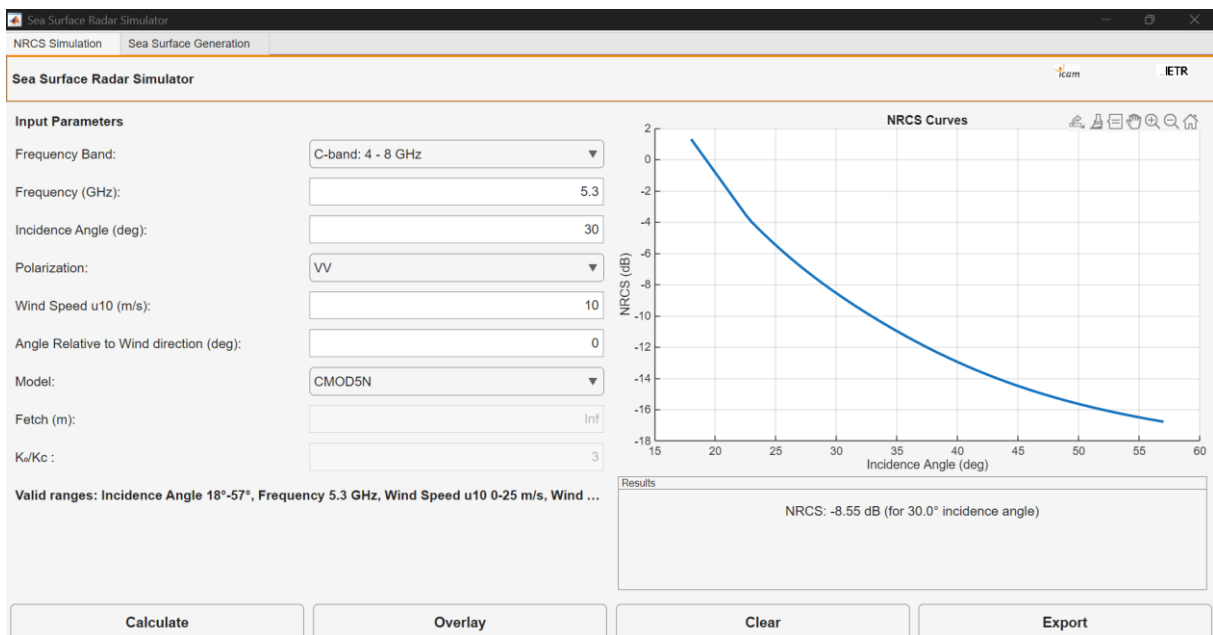


Figure 1: Onglet NRCS

a. Section paramètres (à gauche)

La zone de gauche utilise un `uix.VBox` avec des hauteurs fixes pour un bon alignement. Chaque paramètre est présenté sous la forme `label + composant` dans un `uix.HBox`.

1. **Bande de fréquences** : Dropdown (bande déroulante) proposant les 10 bandes classiques (UHF à W-band) avec leurs plages en GHz. Le callback `bandChangedCallback` extrait automatiquement la fréquence centrale (moyenne des bornes) et l'affecte au champ fréquence. Il appelle également `updateModelList` pour ne proposer que les modèles compatibles avec la bande sélectionnée .
2. **Fréquence (GHz)** : Champ numérique éditable permettant de choisir la valeur exacte de la fréquence. Une validation ultérieure vérifie qu'il reste dans la bande choisie (ajustement silencieux si nécessaire).
3. **Angle d'incidence (deg)** : Valeur unique utilisée pour l'affichage d'une valeur spécifique de la NRCS.
4. **Polarisation** : Dropdown HH / VV / HV. Certains modèles ne supportent que VV (notamment des modèles en bande C et X) , la plupart supportent les polarisations VV et HH. Seuls quelques modèles supportent la polarisation HV (le modèle empirique en bande L Meissner et le modèle statistique TSM).
5. **Vitesse du vent u_{10} (m/s) et angle relatif au vent (deg)** : Champs numériques avec limites.
6. **Modèle** : Dropdown dont les items sont mis à jour dynamiquement. Le callback `modelChangedCallback` :
 - Active/désactive le champ `fetch` pour TSM, SSA1, GO1sh, SPM1.
 - Active/désactive le champ `K0/Kc` uniquement pour TSM (valeur par défaut 4) et GO1sh (valeur par défaut 3).
 - Affiche une note détaillée de validité (plages d'angles, fréquences, vent).
7. **Fetch (m) et K₀/K_c** : Champs conditionnels expliqués ci-dessus.

b. Section affichage et résultats (à droite)

- Un `uiaxes` occupant la majeure partie de l'espace pour tracer la courbe σ^0 (en dB) en fonction de l'angle d'incidence.
- Un panneau « Results » contenant :
 - Le label principal affichant la valeur de σ^0 (en dB) à l'angle d'incidence saisi (interpolation au point le plus proche).
 - Un label d'avertissement orange pour signaler les approximations lors de la conversion vitesse du vent – état de mer

c. Boutons

L'onglet comporte 4 boutons :

```
% Section boutons en bas
btnPanel = uix.HBox('Parent', mainLayoutNRCS, 'Padding', 5, 'Spacing', 10);
calcBtn = uibutton('Parent', btnPanel, 'Text', 'Calculate', 'ButtonPushedFcn',
@(btn,event) calculerCallback());
overlayBtn = uibutton('Parent', btnPanel, 'Text', 'Overlay', 'ButtonPushedFcn',
@(btn,event) superposerCallback());
clearBtn = uibutton('Parent', btnPanel, 'Text', 'Clear', 'ButtonPushedFcn',
@(btn,event) clearCurvesCallback());
exportBtn = uibutton('Parent', btnPanel, 'Text', 'Export', 'ButtonPushedFcn',
@(btn,event) exporterCallback());
```

- Calculate : exécute le calcul complet après toutes les validations.
- Overlay : affiche toutes les courbes stockées avec une légende interactive.
- Clear : réinitialise tout.
- Export : propose l'export de la dernière courbe au format .csv ou de toutes les courbes superposées (.csv multi-colonnes).

3. Onglet « Sea Surface Generation »

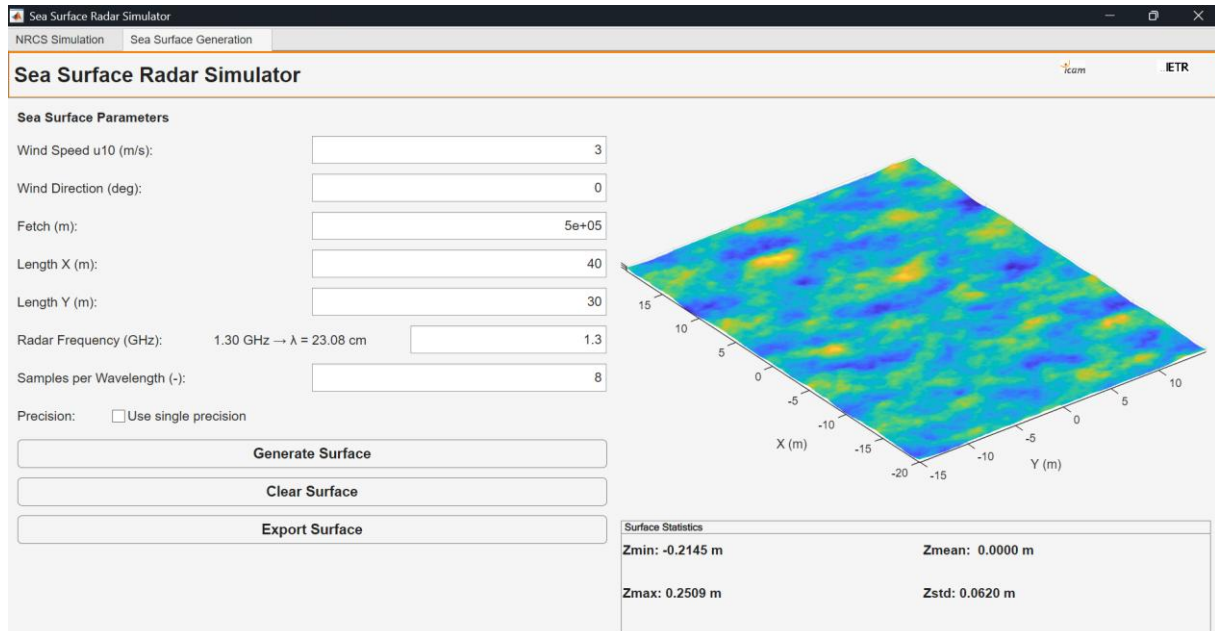


Figure 2: Onglet SeaSurface

a) Paramètres

Cet onglet, lui, contient les paramètres suivants :

- Vitesse et direction du vent, fetch.

- Dimensions physiques de la surface (Lx, Ly en mètres).
- Fréquence radar → un label adjacent affiche en temps réel la longueur d'onde correspondante en cm
- Nombre d'échantillons par longueur d'onde (défaut 8).
- Checkbox pour choisir ou non la précision single (réduction mémoire d'un facteur 2).

b) Affichage

Un uiaxe dédié à la visualisation 3D est utilisée pour afficher la surface de mer une fois qu'elle a été générée.

c) Boutons

Les boutons de cet onglet sont au nombre de 3 :

```
% Boutons pour générer et effacer la surface
generateBtn = uibutton('Parent', paramsSeaSurfacePanel, 'Text', 'Generate Surface', 'ButtonPushedFcn', @(btn, event) generateSeaSurfaceCallback());
clearSurfaceBtn = uibutton('Parent', paramsSeaSurfacePanel, 'Text', 'Clear Surface', 'ButtonPushedFcn', @(btn, event) clearSeaSurfaceCallback());
exportSurfaceBtn = uibutton('Parent', paramsSeaSurfacePanel, 'Text', 'Export Surface', 'ButtonPushedFcn', @(btn, event) exportSeaSurfaceCallback());
```

- Generate Surface : lance la génération après contrôles mémoire.
- Clear Surface : libère la mémoire occupée
- Export Surface : menu complet (.mat, .csv, .png ou les trois) pour exporter la courbe générée.

III. IMPLEMENTATION DES FONCTIONNALITES CLES

1. Calcul de la NRCS : Principe et Structure de calculerCallback

La fonction calculerCallback est le cœur de l'onglet NRCS. Elle gère l'ensemble du processus de calcul d'une courbe $\sigma^0(\theta)$ pour un jeu de paramètres donné. Son exécution suit une séquence rigoureuse et très structurée afin de garantir à la fois la précision scientifique et la robustesse logicielle.

Étape 1 : Récupération et validation initiale des paramètres : Tous les paramètres sont lus depuis les composants stockés dans app.handles : bande, fréquence, angle d'incidence, polarisation, vitesse du vent u_{10} , angle relatif au vent, modèle sélectionné, fetch et K_σ/K_c (si activés). Plusieurs validations sont effectuées immédiatement :

- L'angle relatif au vent est forcé dans $[0^\circ ; 360^\circ]$.
- La fréquence saisie est comparée à la bande sélectionnée ; si elle est hors plage, elle est automatiquement ramenée à la limite la plus proche et un avertissement est affiché via *showWarning*.

```

% Récupère les paramètres via les handles
band = app.handles.bandDrop.Value; % Bande de fréquence sélectionnée
freq = app.handles.freqEdit.Value; % Fréquence en GHz
incidenceAngle = app.handles.angleEdit.Value; % Angle d'incidence
pol = app.handles.polDrop.Value; % 'HH', 'VV', ou 'HV'
windSpeed = app.handles.windSpeedEdit.Value; % Vitesse du vent en m/s
windAngle = app.handles.windEdit.Value; % Angle relatif au vent
model = app.handles.modelDrop.Value; % Modèle choisi
fetch = app.handles.fetchEdit_NRCS.Value; % Fetch du premier onglet pour
les modèles statistiques
k0SurKc = app.handles.k0kcEdit.Value; % K0/Kc pour les modèles statistiques

```

Étape 2 : Conversion vent → état de mer pour les modèles semi-empiriques : Les modèles NRL, TSC, SITPROP, GIT, RRE et HYBRID fonctionnent avec un état de mer entier (Sea State 0 à 7). Une conversion approximative est réalisée : $\text{Sea State} \approx \text{round}((u_{10} / 3.16)^{1.25})$. Un avertissement est généré si la conversion inverse présente une erreur > 0.5 m/s, informant l'utilisateur que l'état de mer est une approximation.

```

% Conversion de la vitesse du vent en état de mer pour les modèles semi-
empiriques
seaState = 0;
semiEmpiricalModels = {'NRL', 'TSC', 'SITPROP', 'GIT', 'RRE', 'HYBRID'};
if ismember(model, semiEmpiricalModels)
    seaState = round((windSpeed / 3.16)^1.25);
    if seaState < 0 || seaState > 7
        showWarning(sprintf('Warning: Converted Sea State (%d) is out of
range (0-7). Using closest valid value.', seaState));
        seaState = max(0, min(7, seaState));
    end
    if abs(windSpeed - 3.16 * (seaState^(1/0.8))) > 0.5
        app.handles.warningLabel.String = sprintf('Warning: Approximated
Sea State (%d) from Wind Speed (%.1f m/s) may not be exact.', seaState,
windSpeed);
    else
        app.handles.warningLabel.String = '';
    end
end

```

Étape 3 : Définition du vecteur d'angles d'incidence θ : Chaque modèle possède sa propre plage de validité en angle d'incidence. Un vecteur θ adapté est créé (pas variable selon le modèle : 0.5° pour la plupart, 0.1° pour certains angles rasants). Si l'angle d'incidence saisi par l'utilisateur est en dehors de la plage du modèle choisi, un avertissement bloque le calcul.

```

% Définition de theta (angle d'incidence) avec plage valide pour chaque modèle
switch model
    case 'NRL'
        theta = 30:0.5:89.9; % Vecteur d'angles d'incidence pour NRL
        if incidenceAngle < 30 || incidenceAngle > 89.9
            showWarning('Warning: Incidence Angle must be between 30° and
89.9° for NRL model.');
```

Étape 4 : Calcul proprement dit de la NRCS selon le modèle : Un switch dirige vers le code spécifique de chaque modèle.

- Pour les modèles semi-empiriques et empiriques : appel direct à des fonctions dédiées (ex. : `f_Nrcs_3dMono_EmpExpSea_Cband_Cmod5N`).

```

    case 'RRE'
        % Validation des plages pour RRE
        c_ms = 3e8;
        f_GHz = c_ms / lambda_m * 1e-9;
        if f_GHz < 9 || f_GHz > 10
            showWarning('Warning: RRE model valid only for X band (9-10
GHz).');
```

- Pour les modèles physiques :
 - Calcul de la permittivité relative complexe de l'eau de mer via f_Sea_ErOmg (SST = 15 °C, SSS = 35 ppt).
 - Appel aux fonctions TSM, SSA1, GO1sh ou SPM1 avec les paramètres avancés (fetch, K_o/K_c).
 - Gestion fine des erreurs (try/catch) et messages spécifiques (ex. : fichier de corrélation manquant pour SSA1).

```

case 'TSM'
    % Paramètres physiques
    sst_C = 15;
    sss_ppt = 35;
    k0SurKc_TSM = app.handles.k0kcEdit.Value;

    % Calcul préliminaire
    F_Hz = freq * 1e9;

    % Permittivité
    [er_complex, ~, K_EM, Kc_TSM] = f_Sea_ErOmg(F_Hz, sst_C, sss_ppt,
windSpeed, fetch, k0SurKc_TSM);
    er2 = er_complex;

    try
        %theta, windAngle, er2, windSpeed, fetch, K_EM, Kc_TSM

        [nrCS_VV_lin, nrCS_HH_lin, nrCS_VH_lin, ~, ~, AddTerm_VV,
AddTerm_HH] = ...

f_NrCS_3dMono_StatisticSea_TSM_Elfo_Gauss_Fetch_v2(theta, windAngle, er2,
windSpeed, fetch, K_EM, Kc_TSM);

        switch pol
            case 'VV'
                nrCS = nrCS_VV_lin + AddTerm_VV;
            case 'HH'
                nrCS = nrCS_HH_lin + AddTerm_HH;
            case 'HV'
                nrCS = nrCS_VH_lin;
        end

        catch ME
            showWarning(['TSM error: ' ME.message]);
            nrCS = zeros(size(theta));
            return;
        end

```

Étape 5 : Conversion en dB et stockage : Les modèles physiques qui retournent σ^0 en échelle linéaire ont convertis en dB via la formule $10 \cdot \log_{10}(\sigma^0)$.

Étape 6 : Affichage : La courbe de σ^0 est tracée simplement avec *plot*, accompagnée d'une grille, d'un titre et de labels clairs. La valeur de σ^0 correspondant à l'angle d'incidence saisi est ensuite calculée et affichée directement dans l'interface.

```
% Conversion de NRCS linéaire en dB pour les modèles linéaires
if ismember(model, {'Quilfen', 'CMOD2-I3', 'CMOD5N', 'Isoguchi', 'XMOD1R',
'XMOD2N', 'Meissner', 'Wentz84', 'Wentz99', 'Masuko_X', 'Masuko_Ka', 'TSM',
'SSA1', 'GO1sh', 'SPM1'})
    nracs_dB = 10 * log10(max(eps, nracs)); % Conversion pour les modèles
linéaires
else
    nracs_dB = nracs;
end
```

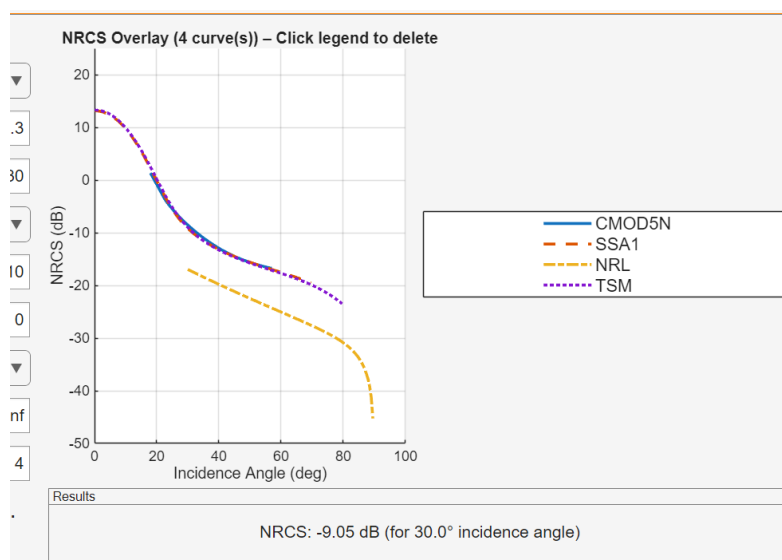
2. Superposition des courbes : superposerCallback

Implémentation d'une légende interactive : chaque entrée est cliquable (ItemHitFcn) et déclenche deleteCurveFromLegend, qui supprime à la fois la ligne graphique et l'entrée correspondante dans app(handles.curves). Le titre et la légende se mettent à jour automatiquement.

Adaptation intelligente des limites Y Une logique spécifique détecte la présence d'une courbe GO1sh :

- Calcul théorique du maximum possible à partir de $10 \cdot \log_{10}(1/(2 \cdot \text{sig_sx} \cdot \text{sig_sy}))$;
- Si polarisation HH présente → limite basse fixée à -80 dB (sinon -50 dB) ;
- Limite haute arrondie au dixième supérieur pour un affichage clair.

DataTips enrichis Chaque courbe possède un modèle de DataTip personnalisé affichant modèle, polarisation, fréquence, vent et angle relatif au vent lorsqu'on survole cette dernière.



3. Export des données

Deux fonctions distinctes :

- `exportLastCurve` : export CSV simple de la dernière courbe calculée (colonnes `Theta_deg` ; `NRCS_dB`).
- `exportAllCurves` : export multi-colonnes intelligent des différentes présentes dans l'overlay (4 maximum) avec des noms de colonnes explicites incluant tous les paramètres

Le nom de fichier proposé est généré automatiquement à partir des paramètres pour une traçabilité maximale.

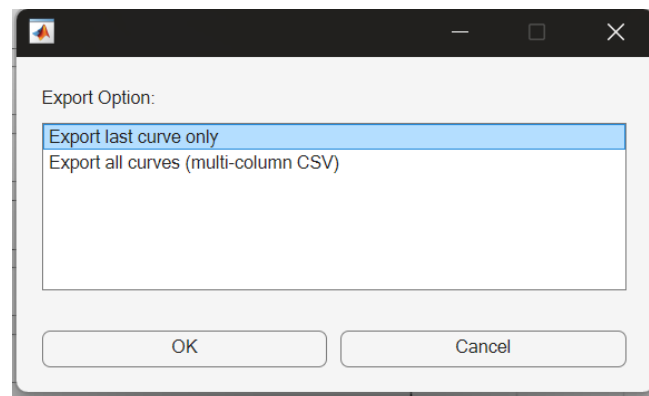


Figure 3: Menu d'export- onglet NRCS

4. Génération de surfaces de mer : `generateSeaSurfaceCallback`

Principe physique : La surface est générée à partir du spectre directionnel unifié d'Elfouhaily et al. [1], utilisant le fetch. La fonction `f_GeneSurfMer3D_Fetch_Elfo_v3` calcule la `IFFT2(...)`, permettant la génération des surfaces .

Contrôles préalables rigoureux :

- Validation des entrées (vent entre 1 et 10 m/s recommandé, `fetch > 0`, dimensions `Lx` et `Ly` positives) ;
- Calcul du pas spatial $dx_y = \lambda / n_{lb}$ (longueur d'onde λ déduite de la fréquence radar) ;
- Détection multi-plateforme de la RAM physique disponible (pour Windows : via `memory`, pour macOS/Linux : via commandes système) ;
- Blocage de la génération , avec warning de validation, si la mémoire estimée dépasse 10 % de la RAM totale, avec message explicite indiquant la taille demandée et la limite.

Exécution et affichage :

- Boîte de dialogue de progression lors de la génération de la surface ;
- Stockage complet dans `app.currentSurface` (matrice HH_XY, grilles X/Y, tous les paramètres et métadonnées) ;
- Message final de confirmation avec temps de génération et la mémoire utilisée.

Export surface : Un menu apparaît, permettant de choisir le type de fichier à utiliser pour l'export

- .mat (structure complète, idéale pour reprise MATLAB) ;
- .csv (X;Y;Z séparés par « ; », encodage UTF-8) ;
- .png haute résolution (résolution 600 dpi) ;
- Ou les trois à la fois, avec nom de fichier cohérent.

IV. VALIDATIONS ET TESTS

1. Validation scientifique

Les modèles NRL, TSC, RRE, GIT, HYBRID et SITTROP sont directement disponibles dans la fonction *seareflectivity* et les fonctions associées de la Radar Toolbox. Une comparaison avec celles-ci a été effectuée sur une plage d'angles d'incidence en faisant varier les paramètres d'entrées selon les domaines de validités des modèles

Résultats obtenus :

- **Modèle NRL** : erreur moyenne absolue de l'ordre de 10^{-14} dB (pratiquement nulle, différence due peut-être à la précision numérique).
- **Modèle TSC** : erreur moyenne absolue de l'ordre de 10^{-9} dB (excellent accord).
- **Modèle RRE** : erreur moyenne absolue de l'ordre de 10^{-15} dB (accord quasi parfait).
- **Modèle GIT** : Bonne correspondance pour les fréquences inférieures à 10 GHz (écarts négligeables). Au-delà de 10 GHz, des écarts significatifs apparaissent, ce modèle n'est donc pas encore entièrement validé.
- **Modèle SITTROP** : Erreur moyenne absolue : **1.2818 dB**, erreur maximale : **7.2827 dB**. Ce modèle également est encore en cours de validation.
- **Modèle HYBRID** : Erreur moyenne absolue : **0.20841 dB**, erreur maximale : **1.769 dB**. Accord relativement bon , mais modèle encore en cours de validation.
- **Modèle CMOD5.N** : Comparaison directe avec la version du site <https://scatterometer.knmi.nl/cmod5/> . Un excellent accord est observé sur la majorité de la plage (18°–57°).

2. Tests fonctionnels

Les tests fonctionnels ont porté sur la robustesse de l'interface et la cohérence des comportements dans des conditions normales.

- **Tests de limites des paramètres** : Vérification que les avertissements apparaissent correctement et que les valeurs sont ajustées ou bloquées (ex. : fréquence hors bande = ajustement automatique, angle d'incidence hors plage du modèle = blocage du calcul avec message clair).
- **Tests de superposition** : Superposition simultanée de 4 courbes issues de modèles très différents (ex. : CMOD5.N + TSM + GO1sh + NRL). Vérification de la stabilité graphique, de la légende interactive (suppression par clic), et de l'adaptation automatique des limites Y (notamment le cas particulier GO1sh avec calcul théorique du maximum).
- **Tests d'activation conditionnelle** : Sélection de TSM/GO1sh → activation correcte des champs fetch et K_o/K_c avec valeurs par défaut appropriées. Retour à un modèle empirique → désactivation et réinitialisation à Inf pour fetch.
- **Tests de génération de surfaces** : Cas (vent 3–10 m/s, dimensions 40×30 m) et cas extrêmes (grande surface, précision single, RAM limite). Vérification du blocage mémoire avec message détaillé et de la reprise correcte après effacement.

Tous les tests fonctionnels ont été passés avec succès, sans plantage ni comportement inattendu.

V. AMELIORATIONS ET LIMITATIONS

1. Limitations actuelles

- Dépendance à des fichiers externes pour SSA1 (fonctions de corrélation 2D)
- Configuration monostatique uniquement
- Problème avec l'affichage de la légende lors de la superposition des courbes, la légende divise l'espace en deux ce qui réduit la taille de l'axe dans lequel sont affichées les courbes

2. Perspectives à court et moyen terme

- Compilation standalone via MATLAB Compiler ou cryptage des fichiers Matlab via les p-codes files.
- Exploration de la parallélisation des tâches grâce à la Parallel Computing Toolbox
- Gestion des problèmes d'affichage de la légende
- Ajout d'un sous-dossier pour le stockage des données exportées.

CONCLUSION

Ce travail a abouti à la réalisation d'un simulateur radar de la surface de mer complet et particulièrement riche en modèles. L'outil dépasse largement le cadre d'une simple calculatrice NRCS : il offre une véritable plateforme d'exploration et de comparaison, avec une interface intuitive, des fonctionnalités avancées (superposition interactive, génération de surfaces) et une attention particulière portée à l'expérience utilisateur et à la fiabilité.

Les nombreux débogages réalisés – notamment sur la stabilité graphique et l'optimisation mémoire – témoignent d'une démarche rigoureuse et ont permis d'obtenir un logiciel mature, même si encore en cours de développement.

Ce simulateur constitue une base solide pour des travaux futurs plus avancés en modélisation radar océanique.

ANNEXES

Annexe A

Le fichier *SeaRadarSimulator.m* est fourni séparément. Il contient l'intégralité du simulateur, y compris toutes les fonctions locales et commentaires.

Il est contenu dans un dossier nommé **Interface** qui renferme tout ce qui est nécessaire pour le bon fonctionnement de l'interface.

Annexe B : Tableau récapitulatif des modèles implémentés

Tableau 1: Détails des modèles implémentés

Modèle	Type	Fréquence principale	Polarisation supportée	Plage θ typique (rasance)	Plage vitesse vent (u_{10} m/s)	Plage angle relatif au vent	Utilisation du Fetch ou non
NRL	Semi-empirique	0.5–35 GHz	HH, VV	30°–89.9°	0–13.2		Non
TSC	Semi-empirique	0.5–35 GHz	HH, VV	0°–89.9°	0–11.4	0°–360°	Non
GIT	Semi-empirique	1–100 GHz	HH, VV	80°–89.9°	3.2–13.2	0°–360°	Non
HYBRID	Semi-empirique (hybride)	0.1–100 GHz	HH, VV	60°–90°	0–11.4	0°–360°	Non
SITTROP	Empirique	9.375 GHz	HH, VV	80°–89.8°	0–14.9	0° ou 90° uniquement	Non
RRE	Semi-empirique	9–10 GHz	HH, VV	80°–89.9°	3.2–13.2	0°–360°	Non
Quilfen	Empirique	5.3 GHz (Bande C)	VV	18°–60°	0–20	0°–360°	Non
CMOD2-I3	Empirique	5.3 GHz (Bande C)	VV	18°–57°	0–25	0°–360°	Non
CMOD5N	Empirique	5.3 GHz (Bande C)	VV	18°–57°	0–25	0°–360°	Non

Isoguchi	Empirique	1.3 GHz (Bande L)	VV, HH	17°–43°	0–20	0°–360°	Non
Meissner	Empirique	1.26 GHz (Bande L)	VV, HH, HV	24.36°– 49.29°	0.2–22	0°–360°	Non
XMOD1R	Empirique	10 GHz	VV	20°–60°	0–20	0°–360°	Non
XMOD2N	Empirique	10 GHz (Bande X)	VV	18°–46°	2–25	0°–360°	Non
Masuko_X	Empirique	10 GHz (Bande X)	VV, HH	30°–60°	3.2–17.2 (u ₁₉)	0°–360°	Non
NSCAT1	Empirique	13.6 GHz (Bande Ku)	VV, HH	15°–60°	1–30	0°–360°	Non
Wentz84	Empirique	14.6 GHz (Bande Ku)	VV, HH	0°–60°	0–20	0°–360°	Non
Wentz99	Empirique	14.6 GHz (Bande Ku)	VV, HH	15°–65°	0–20	0°–360°	Non
Masuko_Ka	Empirique	34.43 GHz (Bande Ka)	VV, HH	30°–60°	3.2–17.2 (u ₁₉)	0°–360°	Non
TSM	Physique (Two-Scale)	0.1–100 GHz	VV, HH, HV	0°–80°	0–25	0°–360°	Oui
SSA1	Physique (SSA ordre 1)	0.1–100 GHz	VV, HH	0°–70°	3–25	0°–360°	Oui
GO1sh	Physique (GO ordre 1)	0.1–100 GHz	VV, HH	0°–50°	0–25	0°–360°	Oui
SPM1	Physique (SPM ordre 1)	0.1–100 GHz	VV, HH	0°–80°	0–25	0°–360°	Oui

Annexe C : Glossaire

app.handles : Structure globale MATLAB utilisée pour stocker tous les composants de l'interface utilisateur (boutons, champs, axes, etc.) ainsi que les données calculées (courbes NRCS, surface de mer). Elle permet d'accéder facilement aux éléments de l'interface depuis n'importe quelle fonction (callback).

Callback : Fonction MATLAB appelée automatiquement lorsqu'un événement se produit dans l'interface (clic sur un bouton, changement de valeur dans un champ, redimensionnement de fenêtre). Exemples : `calculerCallback`, `superposerCallback`, `bandChangedCallback`.

Dropdown (uidropdown) : Composant d'interface graphique (menu déroulant) permettant de choisir une option parmi une liste prédéfinie. Utilisé pour sélectionner la bande de fréquence, le modèle NRCS ou la polarisation.

Fallback : Mécanisme de secours ou de remplacement. Dans le code, il s'agit du texte « ICAM | IETR » affiché si les fichiers des logos (`logo_icam.png`, `logo_ietr.png`) sont absents ou non chargés.

GUI Layout Toolbox (uix) : Toolbox externe (Add-On MATLAB) permettant de créer des mises en page flexibles et responsives sans utiliser App Designer. Elle fournit des conteneurs comme `uix.VBox`, `uix.HBox`, `uix.Grid` pour organiser les composants.

uiaxes : Composant graphique moderne de MATLAB (équivalent de axes classique mais conçu pour les uifigure). Utilisé pour afficher les courbes NRCS et les surfaces 3D.

uibutton : Bouton graphique de l'interface utilisateur. Exemples : « Calculate », « Overlay », « Generate Surface ».

uifigure : Fenêtre principale moderne de MATLAB (équivalent de figure classique mais pour les interfaces App Designer et uifigure-based). C'est la fenêtre racine du simulateur.

uitab / uitabgroup : Onglets (tabs) et groupe d'onglets. Le simulateur utilise un `uitabgroup` contenant deux `uitab` : « NRCS Simulation » et « Sea Surface Generation ».

uix.HBox / uix.VBox : Conteneurs horizontaux (`HBox`) et verticaux (`VBox`) de la GUI Layout Toolbox. Ils permettent d'aligner automatiquement les composants les uns à côté des autres (horizontalement) ou les uns au-dessus des autres (verticalement) avec un espacement et un remplissage contrôlés.

uix.Grid : Conteneur en grille (tableau) de la GUI Layout Toolbox, utilisé ici pour placer l'axe graphique (`uiaxes`) en occupant toute la zone disponible.

uiaalert : Fenêtre pop-up d'alerte ou d'information de MATLAB (similaire à `msgbox` mais plus moderne). Utilisée pour afficher les warnings et messages de succès.

uiprogressdlg : Boîte de dialogue de progression (barre de chargement indéterminée) utilisée pendant la génération de la surface de mer.

NRCS (Normalized Radar Cross Section) / σ^0 : Section efficace radar normalisée (en dB), grandeur physique mesurant la puissance rétrodiffusée par unité de surface de la mer.

Fetch : Distance sur laquelle le vent souffle de façon constante (en mètres ou km). Influence le développement des vagues et donc la rugosité de la surface.

u_{10} : Vitesse du vent à 10 mètres au-dessus de la surface de la mer (en m/s).

Angle d'incidence θ : Angle entre le faisceau radar et la normale à la surface de la mer.

Angle relatif au vent : Angle entre la direction du vent et la direction du faisceau radar (0° – 360°).

Polarisation : Orientation du champ électrique de l'onde radar : VV (verticale émission/réception), HH (horizontale), HV (croisée).

Spectre d'Elfouhaily : Spectre directionnel unifié des vagues de surface, utilisé comme base pour la génération des surfaces réalistes.

REFERENCES

- [1] Elfouhaily, T., B. Chapron, K. Katsaros, and D. Vandemark. "A Unified Directional Spectrum for Long and Short Wind-Driven Waves." *Journal of Geophysical Research: Oceans* 102, no. C7 (July 15, 1997): 15781-96.
<https://doi.org/10.1029/97JC00467>
- [2] Long, Maurice W. *Radar Reflectivity of Land and Sea*, 3rd Ed. Boston: Artech House, 2001.
- [3] Tessendorf, Jerry . "Simulating Ocean Water." Presented at SigGraph, 1999 – 2004
- [4] Antipov, Irina. "Simulation of Sea Clutter Returns." Department of Defence, June 1998
- [5] Ward, Keith D., Simon Watts, and Robert J. A. Tough. *Sea Clutter: Scattering, the K Distribution and Radar Performance*. IET Radar, Sonar, Navigation and Avionics Series 20. London: Institution of Engineering and Technology, 2006
- [6] Masuko, Harunobu, Ken'ichi Okamoto, Masanobu Shimada, and Shuntaro Niwa. "Measurement of Microwave Backscattering Signatures of the Ocean Surface Using X Band and K a Band Airborne Scatterometers." *Journal of Geophysical Research* 91, no. C11 (1986): 13065. <https://doi.org/10.1029/JC091iC11p13065>
- [7] Nathanson, Fred E., J. Patrick Reilly, and Marvin N. Cohen. *Radar Design Principles*. Mendham, NJ: SciTech Publishing, 1999